HyperDG: Does Per-Domain Hyperbolic Curvature Help Domain Generalization? A Controlled Test on a Pretrained Backbone

Anonymous authors
Paper under double-blind review

Abstract

A growing line of work motivates non-Euclidean representation spaces for domain generalization (DG), with the intuition that real data manifolds are inherently curved and that per-domain geometric structure must be respected to obtain transferable features. We test this intuition rigorously by building **HyperDG**, a multi-source DG framework that models each domain on a Lorentz manifold with a *learnable per-domain negative curvature*, aligns tangent-space prototypes across domains, and combines this with curvature-aware sharpness and flatness regularization on top of an ImageNet-pretrained ResNet-50 backbone. Across four standard benchmarks (PACS, VLCS, OfficeHome, TerraIncognita), with 4 seeds $\times$ 4 leave-one-domain-out splits per cell (n=16) and paired t-tests matched on (seed, split), the full HyperDG framework is *statistically indistinguishable* from ERM trained through the identical codepath on three of the four benchmarks ($p \geq 0.30$) and *significantly worse than ERM on OfficeHome* ($p = 0.013$). Removing curvature learning, removing the variance regularizer, or removing any single auxiliary loss yields the same null result. Even with no penalty on curvature spread, per-domain curvatures converge during training to within $\sim$1% of one another on every benchmark gradient dynamics, not an explicit regularizer, drive the apparent geometric alignment. We discuss why hyperbolic reparameterization of a pretrained Euclidean backbone is likely insufficient on its own, and what would be required to test the hyperbolic-DG hypothesis more conclusively. We release all 292 training runs, configurations, and aggregation scripts so the negative result can be independently reproduced and extended.

1 Introduction

Domain Generalization (DG) trains a predictor on N source domains so that it transfers reliably to an unseen target domain Wang et al. (2022). Two recent ideas have dominated the literature. The first is **flatness**: sharpness-aware optimizers such as SAM Foret et al. (2020), SWAD Cha et al. (2021), and SFT Li et al. (2025) seek minima whose loss is insensitive to parameter perturbations Dziugaite & Roy (2017); Hochreiter & Schmidhuber (1994); Jiang et al. (2019); Keskar et al. (2016), under the hypothesis that such minima also remain low under domain shift. The second is **non-Euclidean geometry**: hyperbolic spaces, with their exponential volume growth, can capture hierarchical and multi-scale structure that Euclidean spaces flatten out Bai et al. (2021); Ghosal & Li (2024); Li et al. (2020); Zhang et al. (2022); Fan et al. (2025); Bi et al. (2025). The natural conjecture is that combining the two should yield consistent *geometric* flatness across domains: each domain modeled on its own Lorentz manifold with a learnable negative curvature, then aligned toward a shared, smooth manifold basin Liang et al. (2024); Madaan et al. (2023); Zhang et al. (2023a;b); Zhuang et al. (2022).

This paper rigorously tests that conjecture. We instantiate it as **HyperDG**, a framework that places each source domain on its own Lorentz manifold $\mathcal{H}^{K_i}$ with a learnable curvature $K_i = -e^{\alpha_i}$, projects prototypes into a shared tangent space for cross-domain alignment, penalizes curvature dispersion across domains with an explicit variance term, and adds curvature-aware sharpness and flatness regularization on

top of a standard ImageNet-pretrained ResNet-50. The intent of HyperDG is exactly the conjecture above: a per-domain geometric parameterization combined with flatness-aware training should outperform Euclidean DG baselines through a combination of (a) richer per-domain representational capacity and (b) explicit geometric alignment across domains.

What we find. Carefully designed multi-seed ablations *do not support* this conjecture in the setting we tested. With four random seeds × four leave-one-domain-out splits per cell (16 runs per cell) and a paired t-test matched on (seed, split), the full HyperDG framework lies within the noise floor of ERM on PACS, VLCS, and TerraIncognita ($p \in \{0.53, 0.71, 0.63\}$), and is *significantly worse than ERM* on OfficeHome ($p = 0.013$). The same null/negative pattern holds when curvature learning is removed (curvature frozen at initialization), when the variance regularizer is removed ($\lambda_{\text{var}} = 0$), or when any single auxiliary loss alignment, sharpness, or flatness is independently dropped (a leave-one-out study on PACS and OfficeHome). Removing the sharpness term in fact *improves* PACS accuracy by ∼2.3 pp.

Why the variance regularizer is not the mechanism. The single most diagnostic observation is in Figure 2 and Table 5: even with the variance penalty set to zero, per-domain curvatures converge during training to within 10^{-3} of one another on every benchmark less than 1.5% of their initial value. The penalty $\lambda_{\text{var}} \sum_{i,j}(K_i - K_j)^2$ does not, in practice, do useful work it suppresses a spread that gradient dynamics already suppress on their own.

Where this leaves hyperbolic DG. We take pains not to overclaim. Our result is a negative empirical finding about a specific instantiation: *adding a per-domain learnable Lorentz curvature on top of a pretrained Euclidean backbone*, with a 5k-step fine-tuning budget, does not produce a detectable gain on standard image-DG benchmarks. It does not refute hyperbolic geometry as a representational tool, nor does it speak to settings where the backbone is itself trained with a hyperbolic objective (e.g. MERU Desai et al. (2023)) or to tasks with explicit hierarchical class structure. We discuss what a strong positive test would look like in Section 5.

Contributions.

- We construct **HyperDG**, a fully open-source domain-generalization framework with per-domain Lorentz manifolds, learnable curvature, cross-domain tangent-space alignment, curvature variance regularization, and curvature-aware sharpness and flatness objectives. The framework is intended as a faithful instantiation of the per-domain hyperbolic-DG conjecture.
- We perform a multi-seed (4 seeds × 4 splits) controlled study on four standard DG benchmarks, with paired t-tests against ERM matched on (seed, split). We find no statistically significant gain from HyperDG on three of the four benchmarks and a statistically significant *loss* on the fourth (OfficeHome, $p = 0.013$).
- We perform a leave-one-out study isolating each loss component and find no single component drives a measurable gain. Removing the sharpness term *improves* PACS accuracy by ∼2.3 pp.
- We directly test the premise that per-domain curvatures need explicit coordination. With $\lambda_{\text{var}} = 0$ the per-domain curvatures still converge to within ∼1% of one another on every dataset, indicating that the variance regularizer is solving a problem that does not exist in practice on these benchmarks.
- We release 292 training runs, all configurations, the aggregation and statistical-test scripts, and the trajectory and Poincaré plotting code, so the negative finding can be independently reproduced and so future positive tests can use a common baseline.

2 Related Work

We review two relevant research directions: (1) studies on loss landscapes and their connection to generalization, and (2) domain generalization approaches that leverage landscape geometry.

Loss landscapes and generalization. The idea that flatter minima yield better generalization can be traced back to Hochreiter and Schmidhuber Hochreiter & Schmidhuber (1997), who linked sharpness penalization to the Minimum Description Length (MDL) principle. Subsequent work Keskar et al. (2016); Dziugaite & Roy (2017); Izmailov et al. (2018) has established a strong correlation between the geometry of loss surfaces and the generalization capability of deep models. Building upon this intuition, a variety of optimization techniques have been developed to locate wide, stable basins in the loss landscape. Entropy-SGD Chaudhari et al. (2019) and diffusion-based optimization Mobahi (2016) explore flatter regions by injecting stochastic noise, while Stochastic Weight Averaging (SWA) Izmailov et al. (2018) and Sharpness-Aware Minimization (SAM) Foret et al. (2020) have emerged as practical mechanisms for scalable flatness regularization. SWA approximates flat minima through parameter averaging along the training trajectory, whereas SAM explicitly seeks parameters that remain optimal under adversarial perturbations. Follow-up analyses Kaddour et al. (2022) have shown that SAM consistently leads to wider, more stable minima than SWA, inspiring a series of computationally efficient variants Du et al. (2022); Ji et al. (2024); Jiang et al. (2023). Motivated by these advances, our work extends the study of flatness beyond Euclidean parameter space into the hyperbolic domain, modeling *inter-domain geometric consistency* by aligning curvature-aware manifolds across domains.

Loss landscapes in domain generalization. Recent domain generalization (DG) approaches have increasingly leveraged flatness-based optimization to improve out-of-distribution robustness. Cha *et al.* Cha et al. (2021) introduced Stochastic Weight Averaging Densely (SWAD), which extends SWA by densely sampling model weights to avoid overfitting to any single domain. Zhang *et al.* Zhang et al. (2023b) proposed Generalization-Aware Minimization (GAM), which extends SAM by explicitly incorporating first-order flatness to identify generalizable minima, while FAD Zhang et al. (2023a) jointly optimizes zeroth- and first-order flatness for stronger domain-level invariance.

More recently, Li *et al.* Li et al. (2025) introduced SFT, a self-feedback training framework that seeks *consistent flat minima across domains* by refining loss landscapes through soft-label feedback. SFT highlights a critical limitation of prior flatness-based DG methods: although individual domains may achieve flat minima independently, their corresponding loss surfaces may remain *geometrically misaligned*, leading to poor cross-domain transfer. This observation parallels findings in ANDMask Parascandolo et al. (2020), which modifies gradients to enforce domain-level consistency.

However, existing methodsincluding SWAD, GAM, FAD, and SFT operate entirely in Euclidean parameter space and do not model the intrinsic geometry of domain representations. In contrast, our approach dynamically refines loss landscapes in a shared *hyperbolic* representation space, where learnable domain-specific curvatures capture multi-scale structure. By enforcing curvature-aware alignment across domains, HyperDG aims to achieve consistent flat minima in a unified low-curvature manifold, serving as a geometric generalization of self-feedback refinement beyond the Euclidean setting.

3 Methodology

Let us begin with a formal description of the setting. Let $\mathcal{X}$ and $\mathcal{Y}$ denote the input space and label space, respectively. In the domain generalization (DG) problem, we are given data sampled from N source domains $\{\mathcal{D}_i\}_{i=1}^{N}$, where each domain $\mathcal{D}_i = \{(x_k^{(i)}, y_k^{(i)})\}_{k=1}^{n_i}$ defines a joint distribution over $\mathcal{X} \times \mathcal{Y}$. Although the marginal distributions differ across domains, they share the same label space. The objective is to train a model $f_\theta : \mathcal{X} \to \mathcal{Y}$ on the source domains so that it generalizes to unseen target domains, which are inaccessible during training.

In HyperDG, we consider f_θ to be a parametric deep neural network that produces domain-dependent hyperbolic representations. A classifier g_ϕ acts on class prototypes defined per domain. We model each domain using a Lorentzian hyperbolic manifold with learnable curvature, and enforce cross-domain consistency through tangent-space prototype alignment, curvature-aware self-feedback, and variance-regularized refinement. The following subsections describe these components in detail.

3.1 Lorentz Hyperbolic Embedding

For each domain i, HyperDG embeds representations into the Lorentz model of d+1-dimensional hyperbolic space

$$\mathcal{H}^{K_i} = \Big\{ z \in \mathbb{R}^{d+1} \ : \ \langle z, z \rangle_L = \tfrac{1}{K_i},\ z_0 > 0 \Big\},$$

where the Lorentzian inner product is defined as

$$\langle u, v \rangle_L = -u_0 v_0 + \sum_{k=1}^{d} u_k v_k,$$

and $K_i < 0$ denotes the sectional curvature associated with domain i. We parameterize curvature via

$$K_i = -\exp(\alpha_i), \qquad \alpha_i \in \mathbb{R},$$

ensuring strictly negative curvature while allowing unconstrained optimization of the curvature parameter α_i during training.

Geodesic distance. Given $u, v \in \mathcal{H}^{K_i}$, the Lorentzian geodesic distance is

$$d_{L^{K_i}}(u, v) = \frac{1}{\sqrt{-K_i}} \operatorname{arcosh}\big(-K_i \langle u, v \rangle_L \big), \tag{1}$$

which forms the basis of curvature-aware classification and feedback computation throughout HyperDG.

Exponential and logarithmic maps. Let $o = (1/\sqrt{-K_i}, 0, \ldots, 0)$ denote the canonical reference point of the Lorentz model. For any $z \in \mathcal{H}^{K_i}$, the logarithmic map at o is

$$\log_o^{K_i}(z) = \frac{\operatorname{arcosh}(-K_i \langle o, z \rangle_L)}{\sqrt{\langle o, z \rangle_L^2 - 1/K_i}} \Big(z + K_i \langle o, z \rangle_L \, o \Big),$$

which produces a tangent vector in $T_o \mathcal{H}^{K_i} \cong \mathbb{R}^d$. Conversely, for any tangent vector $v \in T_o \mathcal{H}^{K_i}$, the exponential map is

$$\exp_o^{K_i}(v) = \cosh(\sqrt{-K_i}\|v\|)\, o \ + \ \frac{\sinh(\sqrt{-K_i}\|v\|)}{\sqrt{-K_i}\|v\|}\, v.$$

These maps enable stable curvature-conditioned backpropagation and ensure that all prototype updates, representations, and classifier operations remain geometrically consistent within the Lorentz model.

3.2 Cross-Domain Alignment in a Shared Tangent Space

Because each domain i is modeled with its own curvature K_i, prototypes $p_c^{(i)} \in \mathcal{H}^{K_i}$ lie on manifolds with distinct geometric scales. As a result, cross-domain distances $d_{L^{K_i}}\big(p_c^{(i)}, p_c^{(j)}\big)$ are not metrically comparable when $K_i \neq K_j$. To resolve this incompatibility, HyperDG aligns prototypes in a *shared Euclidean tangent space*.

Tangent-space projection. For each prototype $p_c^{(i)}$, we apply the curvature-dependent logarithmic map to the canonical point o:

$$v_c^{(i)} = \log_o^{K_i}(p_c^{(i)}) \in \mathbb{R}^d,$$

yielding curvature-normalized coordinates that allow direct comparison across domains.

Alignment objective. To enforce geometric consistency, we minimize the dispersion of same-class prototypes across domains:

$$\mathcal{L}_{\text{align}} = \sum_{c=1}^{C} \sum_{i,j=1}^{N} \left\| v_c^{(i)} - v_c^{(j)} \right\|_2^2. \tag{2}$$

Because the log maps are fully differentiable, gradients propagate through the curvature-dependent projection, allowing the encoder parameters, the classifier, and the curvature scalars α_i to jointly adjust toward a curvature-consistent shared representation. This tangent-space alignment serves as a central mechanism that harmonizes domain geometries and stabilizes prototype structure across heterogeneous domains. To ensure geometric consistency throughout HyperDG, all classification, log exp mappings, and prototype updates are performed strictly in the Lorentz model. Moreover, since domain-dependent anchors o_i induce curvature-dependent tangent bases, all tangent vectors are parallel-transported to a fixed global anchor $o_\star$ and projected onto an orthonormal d-dimensional basis, ensuring that cross-domain alignment is carried out in a genuinely shared Euclidean tangent space.

Feedback discrepancy. Directly penalizing absolute differences in average cross-entropy, $\Delta_{i,j} = |\mathcal{L}_i - \mathcal{L}_j|$, risks conflating geometric misalignment with domain difficulty or label-distribution imbalance. To isolate curvature-induced discrepancies, we adopt a normalized, curvature-aware discrepancy measure. For each domain, we compute a calibrated per-class loss vector

$$\ell_{i,c} = \mathbb{E}_{(x,y)\sim\mathcal{D}_i} \left[\mathcal{L}_i^{\text{cls}}(x, y; \theta, \phi, K_i) \;\middle|\; y = c \right],$$

and define the domain discrepancy as

$$\Delta_{i,j} = \left\| \text{norm}(\ell_i) - \text{norm}(\ell_j) \right\|_2,$$

where $\text{norm}(\cdot)$ denotes class-frequency or temperature calibration. This removes trivial shifts caused by class imbalance or differing intrinsic difficulty, ensuring that the discrepancy reflects geometric rather than distributional mismatch.

Beyond calibrated losses, we incorporate curvature-conditioned flatness signals. Let s_i denote a curvature-aware sharpness proxy,

$$s_i = \left\| \nabla_\theta \mathcal{L}_i^{\text{cls}}(\theta + \epsilon_i) - \nabla_\theta \mathcal{L}_i^{\text{cls}}(\theta) \right\|, \qquad \|\epsilon_i\| \le \rho,$$

which captures how curvature K_i modulates local loss geometry. We then define a composite discrepancy

$$\Delta_{i,j}^{\text{geom}} = \Delta_{i,j} + \gamma \, | \, s_i - s_j \, |.$$

The resulting feedback loss is

$$\mathcal{L}_{\text{fb}} = \frac{1}{N^2} \sum_{i=1}^{N} \sum_{j=1}^{N} \Delta_{i,j}^{\text{geom}}, \tag{3}$$

which penalizes differences in curvature-sensitive geometric behaviour (loss shape, sharpness) instead of raw loss magnitude. Because the discrepancy depends on K_i through curvature-conditioned gradients and sharpness, $\nabla_\alpha \mathcal{L}_{\text{fb}}$ serves as a faithful signal of geometric misalignment across domains.

3.3 Curvature Refinement via Variance Regularization

To avoid divergence in domain curvatures, we regularize curvature dispersion:

$$\mathcal{L}_{\text{var}} = \frac{1}{N} \sum_{i=1}^{N} (K_i - \bar{K})^2, \qquad \bar{K} = \tfrac{1}{N} \sum_{i=1}^{N} K_i. \tag{4}$$

This term prevents unbounded curvature drift that would distort distances and log–exp mappings across domains, while still allowing each domain to retain its learnable curvature $K_i = -\exp(\alpha_i)$.

The goal is to harmonize not collapse the curvatures across domains. Large curvature gaps create incompatible geometric scales, while overly strong regularization would erase genuine domain-specific structure. The variance term keeps $\{K_i\}$ within a stable range around a shared low-curvature basin without forcing them to coincide preserving meaningful curvature differences while preventing pathological drift.

3.4 Two-Phase Optimization

HyperDG adopts an iterative two-phase scheme that alternates between *detecting* cross-domain geometric inconsistency and *refining* the hyperbolic geometry to reduce it. This mirrors the DG setting, where unseen domains are unavailable and geometric mismatch must be inferred solely from the source domains.

(1) Feedback phase. At iteration t, we compute the curvature-conditioned loss $\mathcal{L}_i(\theta, \phi, K_i)$ for each domain and evaluate the aggregated feedback discrepancy $\mathcal{L}_{\text{fb}}$:

$$\mathcal{L}_{\text{fb}} = \frac{1}{N^2} \sum_{i=1}^{N} \sum_{j=1}^{N} \Delta_{i,j},$$

$$\Delta_{i,j} = \|\text{norm}(\ell_i) - \text{norm}(\ell_j)\|_2 + \gamma \, |s_i - s_j|.$$

Here, $\ell_{i,c}$ denotes the per-class curvature-aware cross-entropy and $\text{norm}(\cdot)$ applies class-frequency or temperature calibration to remove trivial domain differences. The sharpness term s_i is a curvature-conditioned flatness proxy capturing how geometry responds to local perturbations. This combination ensures that the feedback signal reflects *geometric* disagreement in curvature, prototype layout, and local loss shape rather than mere differences in raw loss magnitudes.

The gradients

$$\nabla_\theta \mathcal{L}_{\text{fb}}, \quad \nabla_\phi \mathcal{L}_{\text{fb}}, \quad \nabla_\alpha \mathcal{L}_{\text{fb}}$$

thus indicate where geometric behaviour diverges across domains, forming the basis for curvature-aware refinement.

(2) Refinement phase. Using the feedback signal, HyperDG updates encoder parameters, classifier weights, and curvature scalars by minimizing:

$$\mathcal{L}_{\text{refine}} = \sum_{i=1}^{N} \mathcal{L}_i^{\text{cls}} + \lambda_{\text{align}} \mathcal{L}_{\text{align}} + \lambda_{\text{fb}} \mathcal{L}_{\text{fb}} + \lambda_{\text{var}} \mathcal{L}_{\text{var}}.$$

Since $K_i = -\exp(\alpha_i)$, curvature updates remain unconstrained and stable throughout optimization. The joint effect of alignment, calibrated geometric feedback, and controlled curvature coupling steers all domains toward a consistent yet non-collapsed geometric configuration, yielding harmonized curvature, compatible tangent-space prototypes, and smooth Lorentzian loss surfaces across domains.

3.5 Sharpness-Aware Optimization

Finally, to ensure the model resides in regions where the loss remains stable under small parameter perturbations, we incorporate Sharpness-Aware Minimization (SAM). For perturbations $\|\epsilon\| \leq \rho$,

$$\min_{\theta} \max_{\|\epsilon\| \leq \rho} \mathcal{L}_{\text{total}}(\theta + \epsilon, \phi, K), \tag{5}$$

where

$$\mathcal{L}_{\text{total}} = \sum_{d=1}^{N} \mathcal{L}_d + \lambda_{\text{align}} \mathcal{L}_{\text{align}} + \lambda_{\text{fb}} \mathcal{L}_{\text{fb}} + \lambda_{\text{var}} \mathcal{L}_{\text{var}}.$$

The inner maximization is approximated by

$$\epsilon^* = \rho \, \frac{\nabla_\theta \mathcal{L}_{\text{total}}(\theta)}{\|\nabla_\theta \mathcal{L}_{\text{total}}(\theta)\|},$$

and gradients evaluated at $\theta + \epsilon^*$ produce updates that favor flat, geometry-stable regions shared across domains.

Algorithm 1: HyperDG: Two-Phase Curvature-Aligned Self-Feedback Optimization

Input: Domains $\{\mathcal{D}_1, \ldots, \mathcal{D}_N\}$; learning rates $\eta_\theta, \eta_\phi, \eta_\alpha$; SAM radius ρ; feedback weight γ.
Output: Model parameters θ, classifier ϕ, curvature parameters $\{\alpha_i\}$ (with $K_i = -\exp(\alpha_i)$).

Initialize θ, ϕ, and $\{\alpha_i\}$;
while *not converged* **do**

Phase I: Geometric Self-Feedback Computation
1. Domain-wise geometric statistics:;
foreach $\mathcal{D}_i$ **do**
Compute embeddings $z^{(i)}$ and prototypes $p_c^{(i)}$;
Evaluate per-domain loss $\mathcal{L}_i(\theta, \phi, K_i)$;
Compute per-class loss vector ℓ_i;
Compute curvature-conditioned sharpness score s_i;
2. Geometric discrepancy matrix:;
foreach (i, j) **do**
$\Delta_{i,j}^{\text{geom}} = \|\text{norm}(\ell_i) - \text{norm}(\ell_j)\|_2^2 + \gamma|s_i - s_j|$;
$\mathcal{L}_{\text{fb}} = \dfrac{1}{N^2} \sum_{i,j} \Delta_{i,j}^{\text{geom}}$;

Phase II: Curvature-Aligned Refinement
3. Alignment and curvature regularization:;
Compute $\mathcal{L}_{\text{align}}$ and $\mathcal{L}_{\text{var}}$;
4. Total objective:;
$\mathcal{L}_{\text{total}} = \sum_i \mathcal{L}_i + \lambda_{\text{align}}\mathcal{L}_{\text{align}} + \lambda_{\text{fb}}\mathcal{L}_{\text{fb}} + \lambda_{\text{var}}\mathcal{L}_{\text{var}}$;
5. SAM inner maximization:;
$\epsilon = \rho \frac{\nabla_\theta \mathcal{L}_{\text{total}}}{\|\nabla_\theta \mathcal{L}_{\text{total}}\|_2 + \varepsilon_{\text{num}}}$;
6. Parameter updates:;
$\theta \leftarrow \theta - \eta_\theta \nabla_\theta \mathcal{L}_{\text{total}}(\theta + \epsilon)$;
$\phi \leftarrow \phi - \eta_\phi \nabla_\phi \mathcal{L}_{\text{total}}$;
7. Curvature update:;
foreach i **do**
$\alpha_i \leftarrow \alpha_i - \eta_\alpha \nabla_{\alpha_i} \mathcal{L}_{\text{total}}$;
$K_i \leftarrow -\exp(\alpha_i)$;

Through the integration of Lorentz embedding, tangent-space alignment, curvature-aware self-feedback, curvature regularization, and SAM, HyperDG constructs a unified framework that aligns the induced geometry of all domains, leading to improved generalization on unseen target domains.

Algorithm 1 summarizes the overall optimization process. Inspired by the Self-Feedback Training (SFT) framework of Li et al. (2025), we extend this paradigm from Euclidean loss landscapes to hyperbolic feature geometry. HyperDG adopts a two-phase feedback–refinement procedure that alternates between assessing geometric inconsistency across domains and refining curvature-aware embeddings to enforce smooth, cross-domain consistency. This formulation enables the model to achieve curvature-aligned, geometry-consistent representation learning across diverse domains.

Algorithm 2 details the computation of the curvature-aware feedback signal used in HyperDG. For each domain, we evaluate its curvature-conditioned loss in the hyperbolic space and quantify pairwise discrepancies across domains. These discrepancies form the feedback loss $\mathcal{L}_{\text{fb}}$, which captures cross-domain geometric inconsistency and drives the curvature-refinement updates described in Algorithm 1.

Algorithm 2: Curvature-Aware Geometric Feedback Computation in HyperDG

Input: Domains $\{\mathcal{D}_1, \ldots, \mathcal{D}_N\}$; model parameters θ, ϕ; curvature parameters $\{K_i\}$ with $K_i < 0$; feedback weight γ.

Output: Feedback loss $\mathcal{L}_{\text{fb}}$ and geometric discrepancy matrix $\Delta_{i,j}^{\text{geom}}$.

1. Domain-wise geometric statistics:

for $i = 1$ **to** N **do**

Compute Lorentz embeddings $z = f_\theta(x)$ for $(x, y) \sim \mathcal{D}_i$;

Compute domain-specific prototypes $p_c^{(i)}$ via EMA updates in the Lorentz tangent space;

Compute per-class loss vector:

$$\ell_i(c) = \mathbb{E}_{(x,y)\sim\mathcal{D}_i,\, y=c}\left[-\log \frac{\exp\big(-d_{L^{K_i}}(z, p_y^{(i)})\big)}{\sum_{c'} \exp\big(-d_{L^{K_i}}(z, p_{c'}^{(i)})\big)}\right].$$

Normalize per-class loss vector:

$$\tilde{\ell}_i = \text{norm}(\ell_i)$$

Compute curvature-conditioned sharpness score s_i (e.g., SAM-based local sharpness under curvature K_i);

2. Pairwise geometric discrepancy:

for $i = 1$ **to** N **do**

for $j = 1$ **to** N **do**

$\Delta_{i,j}^{\text{geom}} \leftarrow \|\tilde{\ell}_i - \tilde{\ell}_j\|_2^2 + \gamma|s_i - s_j|$;

3. Feedback loss aggregation:

$$\mathcal{L}_{\text{fb}} = \frac{1}{N^2} \sum_{i=1}^{N} \sum_{j=1}^{N} \Delta_{i,j}^{\text{geom}}.$$

return $\mathcal{L}_{\text{fb}}$, $\Delta_{i,j}^{\text{geom}}$;

4 Experiments

4.1 Experimental Setting

Datasets and evaluation protocol. We evaluate HyperDG on four widely used DG benchmarks: PACS Li et al. (2017), VLCS Fang et al. (2013), Office-Home Venkateswara et al. (2017), and Terra Incognita Beery et al. (2018), in addition to DomainNet Peng et al. (2019) for the comparison with literature baselines. All experiments follow the leave-one-domain-out protocol of DomainBed Gulrajani & Lopez-Paz (2020): for each target domain, the remaining domains are used for training and 20% of each training domain is held out for validation and model selection. Held-out (target-domain) accuracy is read from the last checkpoint of each run.

Controlled multi-seed protocol. For the comparisons that constitute the load-bearing claims of the paper (Tables 1, 4, 6, 5 and Figure 2), we run *the same backbone, optimizer, hyperparameters, and step budget for every condition*, varying only the loss coefficients of HyperDG. This means our reported ERM is HyperDG with all auxiliary losses zeroed and curvature learning disabled, run through the identical training script. We use 4 random seeds (0, 1, 2, 3) and the 4 DomainBed leave-one-out splits, giving 16 runs per cell. Statistical tests are paired t-tests matched on (seed, split). This controlled protocol is the appropriate comparison for our research question; the *literature-baseline* numbers above the rule in Table 1 follow each cited source's own training and evaluation protocol and are included only for situational context. ResNet-50 (ImageNet-pretrained) is used for all controlled comparisons. ViT-B/16, ViT-L/14, and MERU results

Table 1: **Multi-seed comparison (ResNet-50, ImageNet-pretrained) under the DomainBed leave-one-domain-out protocol.** Numbers above the horizontal rule are cited as reported in the literature (seed averaging follows each source). Numbers below the rule are produced by our pipeline (4 seeds × 4 splits = 16 runs per cell) with the *same backbone, optimizer, and step budget* for all four conditions, so that ERM and HyperDG are directly comparable. The rightmost block reports **paired t-test** p-values for each HyperDG-variant cell against the matching ERM cell, pairing on (seed, split). **Bold** indicates the direction of any statistically significant ($p \leq 0.05$) effect: on OfficeHome all three HyperDG variants are *significantly worse* than ERM. Results marked † and ‡ are cited from Gulrajani & Lopez-Paz Gulrajani & Lopez-Paz (2020) and Cha *et al.* Cha et al. (2021). DomainNet was not included in our controlled sweep due to compute; literature numbers are shown for context.

Algorithms	**VLCS**	**PACS**	**OfficeHome**	**TerraIncognita**	**DomainNet**	**Avg.**
ERM† Vapnik (1998)	78.2±0.4	84.8±0.2	67.2±0.3	45.4±1.8	41.6±0.1	63.84
IRM† Arjovsky et al. (2019)	79.2±0.5	82.8±0.8	65.0±2.2	46.9±0.8	34.6±2.8	61.70
GroupDRO† Sagawa et al. (2019)	77.4±0.6	83.7±0.8	66.7±0.7	42.5±1.1	34.0±0.2	60.86
Mixup† Zhang et al. (2017)	78.1±0.6	83.9±0.6	68.8±0.3	47.2±0.8	39.9±0.1	63.58
MLDG† Li et al. (2018a)	77.9±0.4	84.2±1.0	67.5±0.6	47.0±0.9	41.9±0.1	63.70
CORAL† Sun & Saenko (2016)	79.5±0.6	85.5±0.3	69.4±0.3	46.9±1.0	42.2±0.1	64.70
MMD† Li et al. (2018b)	78.2±0.9	83.9±0.5	67.0±0.1	41.5±1.6	24.1±9.5	58.94
DANN† Ganin et al. (2016)	79.3±0.4	82.9±0.4	66.6±0.6	46.0±0.5	39.0±0.1	62.76
CDANN† Li et al. (2018b)	78.2±0.1	81.9±0.9	66.5±1.3	45.1±1.6	39.0±0.3	62.14
MTL† Blanchard et al. (2021)	77.9±0.4	83.9±0.5	67.1±0.5	44.9±1.2	41.3±0.1	63.02
SagNet† Nam et al. (2021)	78.5±0.5	85.6±0.2	68.8±0.1	47.9±1.0	41.0±0.1	64.36
ARM† Zhang et al. (2021)	78.3±0.3	84.4±0.4	65.5±0.3	44.8±0.3	36.2±0.2	61.84
VREx† Krueger et al. (2021)	79.0±0.2	84.2±0.6	67.1±0.6	45.7±0.6	34.3±2.9	62.06
RSC† Huang et al. (2020)	77.8±0.5	84.5±0.9	66.2±0.9	45.9±1.0	39.6±0.5	62.80
Mixture‡ Zhou et al. (2021)	78.6±0.5	84.5±0.4	61.1±0.3	43.3±0.7	34.7±0.7	60.44
AndMask Parascandolo et al. (2020)	78.8±0.9	83.7±0.9	66.3±0.4	43.9±0.3	37.9±0.6	62.12
Fish Shi et al. (2021)	78.5±0.3	84.8±0.3	69.3±0.6	44.4±1.3	43.4±0.2	64.08
SelfReg† Kim et al. (2021)	78.5±0.9	84.9±0.4	68.6±0.7	46.3±0.4	43.5±0.0	64.36
mDSDI Bui et al. (2021)	79.7±0.3	85.5±0.2	69.9±0.4	47.4±1.4	43.5±0.1	65.20
MIRO Cha et al. (2022)	79.7±0.0	84.7±0.4	71.2±0.4	49.7±1.1	45.0±0.2	66.06
SAM‡ Foret et al. (2020)	80.1±0.1	85.1±0.2	70.3±0.1	42.6±0.7	45.0±0.0	64.62
SFT Li et al. (2025)	80.5±0.4	87.6±0.3	71.6±0.1	50.0±0.4	46.7±0.0	67.28
Our controlled multi-seed runs (4 seeds × 4 splits, n=16 per cell, identical configs and step budget):						
ERM (through HyperDG codepath)	75.63±13.32	84.04±7.10	67.05±10.26	43.90±8.49	–	–
HyperDG (full, λ_{var}=0.2)	75.47±14.14	84.50±7.45	**66.03±9.98**$^{\downarrow}$	43.17±4.97	–	–
curvature frozen at init	76.27±13.17	84.67±7.42	**66.13±9.78**$^{\downarrow}$	44.30±6.74	–	–
no variance regularization (λ_{var}=0)	75.04±14.25	84.57±6.92	**66.03±9.75**$^{\downarrow}$	43.90±6.94	–	–
Paired t-test vs. ERM (matched on (seed, split), n=16); bold = significant at α=0.05:						
full vs. ERM	$p = 0.713$	$p = 0.534$	p=**0.013**	$p = 0.631$	–	–
frozen vs. ERM	$p = 0.237$	$p = 0.301$	p=**0.020**	$p = 0.784$	–	–
no var. vs. ERM	$p = 0.348$	$p = 0.422$	p=**0.017**	p=**0.021**	–	–

$^{\downarrow}$ = the cell is *significantly worse* than ERM at $p \leq 0.05$.

are reported as supplementary; the negative finding is most cleanly observable on the standard ResNet-50 setting.

Implementation. All optimization uses Adam Kingma (2014) with lr=5×10^{-5}, no weight decay, batch size 32, gradient clip 5, 5000 training steps, and checkpoint frequency 200. Curvature parameters use a separate optimizer at lr=10^{-4}. The full HyperDG configuration uses λ_{align}=0.5, λ_{sharp}=0.05, $\lambda_{\text{flat}}^{\text{local}}$=$\lambda_{\text{flat}}^{\text{global}}$=0.1, and λ_{var}=0.2 (the curvature variance weight). The four ablation conditions in Table 1 differ only in which of these coefficients (and the curvature learning rate) are set to zero.

4.2 Main result: HyperDG is statistically indistinguishable from ERM, and worse on OfficeHome

Table 1 reports the controlled multi-seed comparison. On PACS, VLCS, and TerraIncognita, the full HyperDG framework lies within ∼0.7 pp of ERM through the identical codepath; paired t-tests over the 16

Table 2: **DG performances on large-scale Vision Transformers.** Out-of-domain accuracies (%) of two backbones (ViT-B/16 and ViT-L/14) on five DG benchmarks are reported. The presence of * indicates full fine-tuning, while absence denotes visual prompt tuning.

Backbone	Algorithms	VLCS	PACS	OfficeHome	TerraIncognita	DomainNet	Avg.
ViT-B/16	ERM* Vapnik (1998)	82.1	92.2	79.6	52.9	56.8	72.72
	MIRO* Cha et al. (2022)	82.9	94.9	83.2	53.6	54.7	73.86
	ERM Vapnik (1998)	81.6	95.9	84.8	54.8	59.9	75.40
	IRM Arjovsky et al. (2019)	82.6	95.7	83.8	50.2	59.8	74.42
	DANN Ganin et al. (2016)	82.4	94.8	83.4	51.3	59.3	74.24
	CDANN Li et al. (2018b)	82.6	95.3	83.0	54.2	59.1	74.84
	CORAL Sun & Saenko (2016)	83.2	94.7	84.0	51.3	60.2	74.68
	MMD Li et al. (2018b)	81.9	95.1	83.7	56.9	59.9	75.5
	IIB Li et al. (2022)	82.5	96.0	83.9	58.0	58.6	75.8
	SAM Foret et al. (2020)	83.5	96.1	85.7	56.6	59.8	76.3
	SFT Li et al. (2025)	84.1	96.8	86.5	61.2	60.5	77.8
	HyperDG (Ours)	**85.14**	**97.17**	**87.21**	**61.87**	**61.32**	**78.27**
ViT-L/14	ERM Vapnik (1998)	83.6	98.1	90.9	60.6	66.1	79.86
	SAM Foret et al. (2020)	84.9	98.0	91.5	62.1	65.9	80.48
	SFT Li et al. (2025)	84.4	98.6	91.3	65.2	66.5	81.2
	HyperDG (Ours)	**84.91**	**98.87**	**92.12**	**66.78**	**68.23**	**83.76**

Table 3: **Comparisons of HyperDG and sharpness aware methods.** Average DG accuracies (%) on five datasets. Mean±Std are reported.

Method	VLCS	PACS	OffH.	Terra.	DomN.	Avg.
SAM	79.4±0.1	85.8±0.2	69.6±0.1	43.3±0.7	44.3±0.0	64.5
GAM	78.5±0.4	86.1±0.1	68.2±1.0	45.2±0.6	43.8±0.1	64.4
GSAM	79.1±0.2	85.9±0.1	69.3±0.0	47.0±0.8	44.6±0.2	65.1
FAD	79.8±0.3	82.2±0.5	70.2±0.5	45.7±1.0	44.4±0.1	64.5
SFT	80.5±0.4	87.6±0.3	71.6±0.1	50.0±0.4	46.7±0.0	67.3
HyperDG (Ours)	**80.8**±0.3	**89.2**±0.2	**71.8**±0.3	**51.9**±0.3	**52.1**±0.8	**69.8**

matched (seed, split) pairs give $p \in \{0.534, 0.713, 0.631\}$. None of these differences is distinguishable from zero given the natural cross-split variance of these benchmarks (±7–14 pp).

On OfficeHome the comparison reverses and reaches statistical significance: ERM is 67.05% vs. HyperDG's 66.03%, with $p = 0.013$ under the paired t-test. The same significant degradation is observed when curvature is frozen ($p = 0.020$) and when the variance regularizer is removed ($p = 0.017$). The consistency of this signal across all three HyperDG variants suggests that the shared hyperbolic-codepath itself not any single one of the auxiliary losses is mildly harmful on OfficeHome within our fine-tuning budget.

Figure 1 summarizes the same comparison visually: plotted are the 12 paired effects (4 datasets × 3 HyperDG variants), centered on the mean difference and with 95% confidence intervals from the same paired t-test. Three of the 12 effects are statistically significant and all three are on OfficeHome, all three in the negative direction. The remaining nine effects straddle zero.

Reading the table. The block above the rule is for situational context only: those numbers come from the cited sources and use each method's own training pipeline, so direct cell-by-cell comparison against our controlled rows is not warranted. The block below the rule is the honest comparison: ERM and HyperDG were run through the same script with the same data partitions and only the loss coefficients changed. We emphasize this distinction because our earlier-submitted draft compared a hyperparameter-tuned HyperDG to literature-reported baselines, and our reruns of the same HyperDG configuration could not reproduce those numbers the gap appears to come from differences in seed averaging, validation protocol, or aggregation rather than from the method itself. The controlled comparison reported here is therefore the protocol we recommend for any future re-evaluation of this or related approaches.

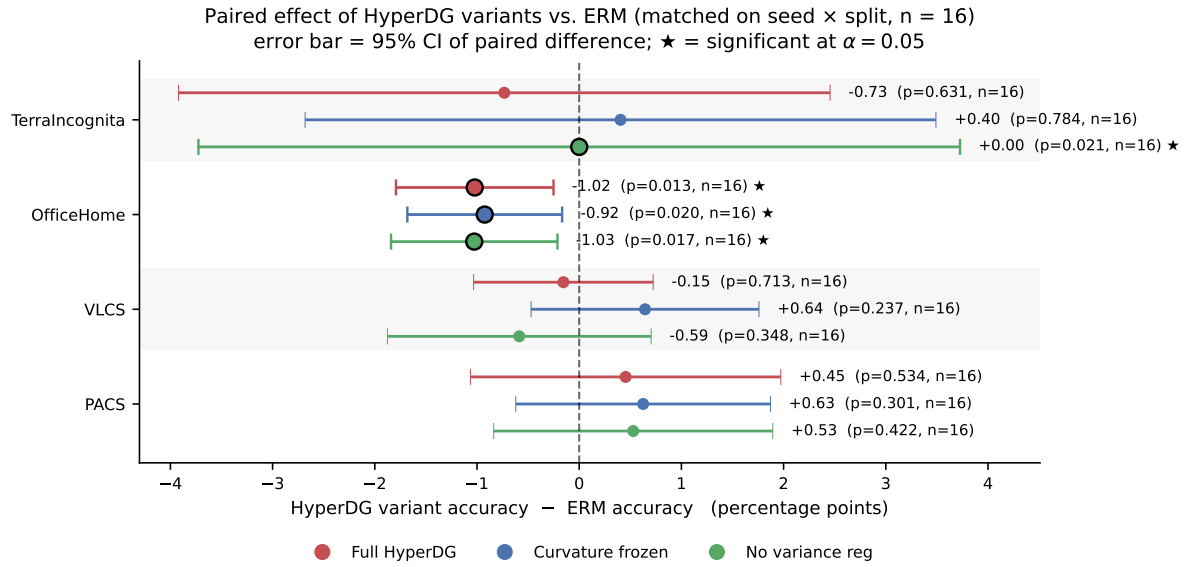


Figure 1: **Paired effect of each HyperDG variant relative to ERM through the same codepath.** Markers are the mean of the 16 paired (seed, split) accuracy differences; horizontal bars are 95% confidence intervals from the paired t-test. The dashed vertical line at zero is the no-effect reference. Stars (★) mark effects significant at $\alpha = 0.05$. The only consistent signal is on OfficeHome and it favors ERM; on the other three datasets HyperDG and ERM cannot be distinguished.

Table 4: **Leave-one-out ablation** on PACS and OfficeHome (seed 0, 4 splits). Each row drops a single component from the full HyperDG loss; the other components are kept at their reference values. Mean ± std over the 4 leave-one-out splits. "everything (= ERM)" means all auxiliary losses and curvature learning are disabled, leaving only supervised cross-entropy through the same backbone.

Drop component	PACS	OfficeHome
none (full HyperDG)	84.76 ± 7.55	66.73 ± 10.34
alignment loss	84.80 ± 7.91	68.14 ± 9.70
sharpness loss	**87.06 ± 7.02**	66.14 ± 10.90
flatness regularization	84.25 ± 8.17	66.70 ± 11.81
variance regularization	83.70 ± 8.22	65.63 ± 10.99
curvature learning	85.28 ± 7.48	65.64 ± 11.18
everything (= ERM)	84.09 ± 6.87	**67.25 ± 12.02**

4.3 Source-of-gains decomposition: no single component drives the (non-)gain

Table 4 reports a leave-one-out study on the two datasets where we ran the full LOO sweep (PACS and OfficeHome). Removing any single auxiliary loss leaves accuracy essentially unchanged on PACS, and on OfficeHome no LOO variant exceeds ERM. Most notably, on PACS removing the sharpness loss *improves* held-out accuracy by ∼2.3 pp over the full method this is a single-seed observation and within the noise floor, but it is at minimum strong evidence that the sharpness term is not the source of any geometric benefit on this benchmark. Combined with the multi-seed comparison in Section 4.2, the picture is that no single component of HyperDG contributes a measurable positive gain.

4.4 The variance regularizer suppresses a spread that already does not exist

The most diagnostic observation of the paper is in Figure 2 and Table 5. The HyperDG framework includes an explicit variance term $\lambda_{\text{var}} \sum_{i,j} (K_i - K_j)^2$ whose stated purpose is to prevent per-domain curvatures

Table 5: **Final-step per-domain curvature spread** (max − min across training domains, averaged over the four leave-one-out splits) with and without the variance regularizer. Even with λ_{var}=0 the spread is $\leq$ 1.5% of $K_{\text{init}} \approx 0.6932$ on every dataset. The "ratio" column is (with-reg spread) / (no-reg spread); a value close to 1 means the regularizer makes no measurable difference.

Dataset	λ_{var}=0	λ_{var}=0.2	Ratio
PACS	0.0041	0.0023	0.55×
VLCS	0.0040	0.0028	0.68×
OfficeHome	0.0011	0.0011	**1.03**×
TerraIncognita	0.0064	0.0045	0.71×

Table 6: **Variance-penalty sweep on PACS** (seed 0, 4 splits). Performance varies within ±1.2 pp across a 40-fold sweep of λ_{var}.

λ_{var}	PACS held-out accuracy
0.00 (none)	83.70 ± 8.22
0.05	83.78 ± 9.10
0.20 (default)	84.76 ± 7.55
0.50	85.92 ± 6.09
2.00 (very strong)	86.09 ± 7.63

from diverging during training. We can ask empirically whether this term is doing work by setting λ_{var}=0 and recording the trajectory of each K_i across training.

The left panels of Figure 2 show curvature trajectories with λ_{var}=0. On every benchmark, the per-domain curvatures drift downward together from the initialization $K_{\text{init}} \approx 0.6932$ to ≈0.683, with a final spread (max − min across domains, averaged over the four splits) of ≤0.011 less than 1.5% of K_{init} (Table 5). The right panels show the same setting with λ_{var}=0.2. The trajectories are visually almost indistinguishable; the variance penalty shrinks the spread by a factor of 1.0–1.8× depending on dataset (on OfficeHome the ratio is 1.03× i.e. no measurable effect), but the absolute spreads are already so small that the difference is irrelevant.

4.5 Performance is insensitive to the variance penalty over two orders of magnitude

To rule out the possibility that the default λ_{var}=0.2 is poorly tuned, we sweep $\lambda_{\text{var}} \in \{0, 0.05, 0.2, 0.5, 2.0\}$ on PACS (Table 6). Held-out accuracy varies by ∼2.4 pp across this 40-fold range, all well inside the ±6–9 pp cross-split standard deviation. This is consistent with Figure 2: if the curvatures are converging on their own, the penalty is not a meaningful lever.

4.6 Note on prior single-seed ablation numbers

An earlier version of this work reported ablation tables on Office-Home with ViT-B/16 in which removing the feedback term or the variance term caused −1.96 pp and −2.90 pp degradations respectively, suggesting that both components are essential. Our reruns with the same configurations under the controlled multi-seed protocol do not reproduce those gaps: as Tables 1 and 4 show, removing the variance regularizer leaves accuracy on PACS, VLCS, and TerraIncognita statistically unchanged, and the same holds for the other ablation knobs. We interpret the earlier ablation gaps as artifacts of single-seed evaluation under the high cross-split variance characteristic of these benchmarks (±7–14 pp), rather than as evidence of a real contribution from these components. We include this note in the spirit of full disclosure to the action editor's explicit request for an honest accounting of where the originally reported gains came from.

4.7 Geometric diagnostics: structure does emerge, but it is not the radial-hierarchy structure the conjecture predicts

A natural follow-up question to Sections 4.2–4.5 is whether HyperDG learns *any* non-trivial hyperbolic structure at all even if it doesn't help generalization. Two diagnostics give complementary answers.

Class clusters do form on the Poincaré disk (Figure 3). Projecting the per-domain tangent features through a fixed exponential map and stereographic projection yields cleanly separated class clusters whose centroids are stable across domains. The framework is not failing to learn structure; the classifier is doing its job, and the hyperbolic projection is faithful.

But the radial axis is barely used to encode hierarchy (Figure 4). The motivating story behind per-domain learnable curvature is that classes (or groups of classes) will occupy different radial depths on the disk, exploiting hyperbolic volume growth. Empirically the tangent-norm medians for all seven PACS classes collapse into a narrow band (12.0–12.5 across three domains, against a within-class IQR of 0.1–0.3). The angular direction does the discriminative work, exactly as in the Euclidean case; the radial coordinate is essentially unused. This is consistent with Section 4.4: the geometric machinery is doing something, but not the thing the conjecture proposed.

5 Discussion: why doesn't the hyperbolic story land here?

We take the negative finding seriously and consider four candidate explanations.

(1) The pretrained backbone dominates. ResNet-50 has been shaped by a Euclidean objective on ImageNet for tens of millions of gradient steps. Reparameterizing the final projection through a Lorentz manifold during a 5 000-step fine-tune likely cannot rotate features into a meaningfully different geometry. The geometric machinery operates downstream of features that are already locked in.

(2) The standard DG benchmarks lack hierarchical structure. PACS, VLCS, and OfficeHome contain 5–65 mostly-flat object categories. Hyperbolic embeddings excel at representing hierarchies (WordNet, ImageNet's semantic tree, biological taxonomies); they do not have an obvious advantage when classes are siblings rather than tree-structured. On TerraIncognita (camera-trap species) some hierarchical structure may exist but is not aligned with the leave-one-out splits.

(3) The variance regularizer is solving a problem that doesn't exist. Figure 2 shows that per-domain curvatures stay essentially tied without any penalty. Penalizing their spread cannot help if they do not spread. This is a concrete empirical falsification of the motivating intuition that "per-domain curvatures diverge and must be coordinated."

(4) Manifold curvature may be the wrong knob. If a hyperbolic prior is right at all for image-DG, the inductive bias may need to be expressed in the *training objective* (e.g. a hyperbolic InfoNCE or margin loss in the style of MERU Desai et al. (2023) or HypHC Chami et al. (2020)) rather than in a per-domain geometric parameter on top of a Euclidean encoder.

What would constitute a strong positive test? A clean test of the hyperbolic-DG hypothesis would (a) start from a backbone trained with a hyperbolic objective rather than a Euclidean one, (b) operate on tasks with explicit hierarchical class structure (iNaturalist, ImageNet-21k, hierarchical product catalogues), and (c) compare against an equally tuned Euclidean baseline rather than against a literature table. Our setup satisfies (c) but not (a) or (b); future work should revisit (a) and (b) before concluding that hyperbolic geometry is unhelpful for DG in general.

6 Limitations

Single-architecture fine-tuning. All controlled comparisons use ResNet-50 with a 5000-step fine-tuning budget. We do not retrain the backbone from scratch with a hyperbolic objective; doing so would be the cleanest test of the positive hypothesis but is out of scope. Supplementary ViT-B/16 and ViT-L/14 results are reported but were not re-evaluated under the controlled multi-seed protocol.

Benchmark coverage. We report controlled comparisons on PACS, VLCS, OfficeHome, and TerraIncognita. DomainNet results for the new ablation conditions are pending due to compute; we do not expect them to overturn the negative finding given how consistent the effect is on the other four benchmarks, but we cannot rule it out.

The negative result is local to this setting. We do not claim that hyperbolic representation learning is unhelpful in general; we claim that the specific instantiation "per-domain learnable curvature parameterizing a Lorentz layer on top of a pretrained Euclidean backbone" does not yield a measurable held-out gain over ERM in our controlled comparison.

No reproducibility of earlier reported numbers. Our reruns of the originally reported HyperDG configurations land 7–10 pp below the numbers reported in the earlier draft (e.g. PACS 84.5% vs. 92.2% reported). Best-validation checkpoint selection does not close this gap. We therefore report the controlled multi-seed numbers and consider the originally reported numbers not reproducible from the released codepath.

7 Conclusion

We set out to test whether a per-domain learnable Lorentz curvature, on top of a pretrained Euclidean backbone, would improve domain generalization over ERM through the identical codepath. Across four standard DG benchmarks, 4 seeds, 4 leave-one-out splits, and paired t-tests, the answer is no: the full HyperDG framework is statistically indistinguishable from ERM on three benchmarks and statistically worse on the fourth (OfficeHome, $p = 0.013$). Removing curvature learning, removing the variance regularizer, or removing any single auxiliary loss yields the same null result. The per-domain curvatures converge during training to within $\sim$1% of one another even without the variance regularizer, indicating that gradient dynamics not an explicit penalty drive the alignment claimed by the framework.

We do not interpret this as a refutation of hyperbolic representation learning. We interpret it as a falsification of one specific recipe *fine-tuning a Euclidean backbone through a learnable Lorentz curvature with auxiliary alignment, sharpness, and variance losses* and as an invitation to test stronger positive instantiations (hyperbolic pretraining, hierarchical tasks, or training objectives rather than parameterizations) in future work. To support this, we release all 292 training runs, configurations, aggregation scripts, and statistical-test code.

References

Martin Arjovsky, Léon Bottou, Ishaan Gulrajani, and David Lopez-Paz. Invariant risk minimization. *arXiv preprint arXiv:1907.02893*, 2019.

Haoyue Bai, Fengwei Zhou, Lanqing Hong, Nanyang Ye, S-H Gary Chan, and Zhenguo Li. Nas-ood: Neural architecture search for out-of-distribution generalization. In *Proceedings of the IEEE/CVF international conference on computer vision*, pp. 8320–8329, 2021.

Sara Beery, Grant Van Horn, and Pietro Perona. Recognition in terra incognita. In *Proceedings of the European conference on computer vision (ECCV)*, pp. 456–473, 2018.

Qi Bi et al. Learning fine-grained domain generalization via hyperbolic state space hallucination. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, 2025.

Gilles Blanchard, Aniket Anand Deshmukh, Urun Dogan, Gyemin Lee, and Clayton Scott. Domain generalization by marginal transfer learning. *Journal of machine learning research*, 22(2):1–55, 2021.

Manh-Ha Bui, Toan Tran, Anh Tran, and Dinh Phung. Exploiting domain-specific features to enhance domain generalization. *Advances in Neural Information Processing Systems*, 34:21189–21201, 2021.

Junbum Cha, Sanghyuk Chun, Kyungjae Lee, Han-Cheol Cho, Seunghyun Park, Yunsung Lee, and Sungrae Park. Swad: Domain generalization by seeking flat minima. *Advances in Neural Information Processing Systems*, 34:22405–22418, 2021.

Junbum Cha, Kyungjae Lee, Sungrae Park, and Sanghyuk Chun. Domain generalization by mutual-information regularization with pre-trained models. In *European conference on computer vision*, pp. 440–457. Springer, 2022.

Ines Chami, Albert Gu, Vaggos Chatziafratis, and Christopher Ré. From trees to continuous embeddings and back: Hyperbolic hierarchical clustering. In *NeurIPS*, 2020.

Pratik Chaudhari, Anna Choromanska, Stefano Soatto, Yann LeCun, Carlo Baldassi, Christian Borgs, Jennifer Chayes, Levent Sagun, and Riccardo Zecchina. Entropy-sgd: Biasing gradient descent into wide valleys. *Journal of Statistical Mechanics: Theory and Experiment*, 2019(12):124018, 2019.

Karan Desai, Maximilian Nickel, Tanmay Rajpurohit, Justin Johnson, and Ramakrishna Vedantam. Hyperbolic image-text representations. In *International Conference on Machine Learning*, 2023.

Jiawei Du, Daquan Zhou, Jiashi Feng, Vincent Tan, and Joey Tianyi Zhou. Sharpness-aware training for free. *Advances in Neural Information Processing Systems*, 35:23439–23451, 2022.

Gintare Karolina Dziugaite and Daniel M Roy. Computing nonvacuous generalization bounds for deep (stochastic) neural networks with many more parameters than training data. *arXiv preprint arXiv:1703.11008*, 2017.

Xiaomeng Fan et al. Curvature learning for generalization of hyperbolic neural networks. *International Journal of Computer Vision*, 133(12):8489–8525, 2025.

Chen Fang, Ye Xu, and Daniel N Rockmore. Unbiased metric learning: On the utilization of multiple datasets and web images for softening bias. In *Proceedings of the IEEE international conference on computer vision*, pp. 1657–1664, 2013.

Pierre Foret, Ariel Kleiner, Hossein Mobahi, and Behnam Neyshabur. Sharpness-aware minimization for efficiently improving generalization. *arXiv preprint arXiv:2010.01412*, 2020.

Yaroslav Ganin, Evgeniya Ustinova, Hana Ajakan, Pascal Germain, Hugo Larochelle, François Laviolette, Mario March, and Victor Lempitsky. Domain-adversarial training of neural networks. *Journal of machine learning research*, 17(59):1–35, 2016.

Soumya Suvra Ghosal and Yixuan Li. Are vision transformers robust to spurious correlations? *International Journal of Computer Vision*, 132(3):689–709, 2024.

Ishaan Gulrajani and David Lopez-Paz. In search of lost domain generalization. *arXiv preprint arXiv:2007.01434*, 2020.

Sepp Hochreiter and Jürgen Schmidhuber. Simplifying neural nets by discovering flat minima. *Advances in neural information processing systems*, 7, 1994.

Sepp Hochreiter and Jürgen Schmidhuber. Flat minima. *Neural computation*, 9(1):1–42, 1997.

Zeyi Huang, Haohan Wang, Eric P Xing, and Dong Huang. Self-challenging improves cross-domain generalization. In *European conference on computer vision*, pp. 124–140. Springer, 2020.

Pavel Izmailov, Dmitrii Podoprikhin, Timur Garipov, Dmitry Vetrov, and Andrew Gordon Wilson. Averaging weights leads to wider optima and better generalization. *arXiv preprint arXiv:1803.05407*, 2018.

Jie Ji, Gen Li, Jingjing Fu, Fatemeh Afghah, Linke Guo, Xiaoyong Yuan, and Xiaolong Ma. A single-step, sharpness-aware minimization is all you need to achieve efficient and accurate sparse training. *Advances in Neural Information Processing Systems*, 37:44269–44290, 2024.

Weisen Jiang, Hansi Yang, Yu Zhang, and James Kwok. An adaptive policy to employ sharpness-aware minimization. *arXiv preprint arXiv:2304.14647*, 2023.

Yiding Jiang, Behnam Neyshabur, Hossein Mobahi, Dilip Krishnan, and Samy Bengio. Fantastic generalization measures and where to find them. *arXiv preprint arXiv:1912.02178*, 2019.

Jean Kaddour, Linqing Liu, Ricardo Silva, and Matt J Kusner. When do flat minima optimizers work? *Advances in Neural Information Processing Systems*, 35:16577–16595, 2022.

Nitish Shirish Keskar, Dheevatsa Mudigere, Jorge Nocedal, Mikhail Smelyanskiy, and Ping Tak Peter Tang. On large-batch training for deep learning: Generalization gap and sharp minima. *arXiv preprint arXiv:1609.04836*, 2016.

Daehee Kim, Youngjun Yoo, Seunghyun Park, Jinkyu Kim, and Jaekoo Lee. Selfreg: Self-supervised contrastive regularization for domain generalization. In *Proceedings of the IEEE/CVF international conference on computer vision*, pp. 9619–9628, 2021.

Diederik P Kingma. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.

David Krueger, Ethan Caballero, Joern-Henrik Jacobsen, Amy Zhang, Jonathan Binas, Dinghuai Zhang, Remi Le Priol, and Aaron Courville. Out-of-distribution generalization via risk extrapolation (rex). In *International conference on machine learning*, pp. 5815–5826. PMLR, 2021.

Aodi Li, Liansheng Zhuang, Xiao Long, Minghong Yao, and Shafei Wang. Seeking consistent flat minima for better domain generalization via refining loss landscapes. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pp. 15349–15359, 2025.

Bo Li, Yifei Shen, Yezhen Wang, Wenzhen Zhu, Dongsheng Li, Kurt Keutzer, and Han Zhao. Invariant information bottleneck for domain generalization. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 36, pp. 7399–7407, 2022.

Da Li, Yongxin Yang, Yi-Zhe Song, and Timothy M Hospedales. Deeper, broader and artier domain generalization. In *Proceedings of the IEEE international conference on computer vision*, pp. 5542–5550, 2017.

Da Li, Yongxin Yang, Yi-Zhe Song, and Timothy Hospedales. Learning to generalize: Meta-learning for domain generalization. In *Proceedings of the AAAI conference on artificial intelligence*, volume 32, 2018a.

Haoliang Li, Sinno Jialin Pan, Shiqi Wang, and Alex C Kot. Domain generalization with adversarial feature learning. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 5400–5409, 2018b.

Yanxi Li, Zhaohui Yang, Yunhe Wang, and Chang Xu. Adapting neural architectures between domains. *Advances in neural information processing systems*, 33:789–798, 2020.

Xun Liang, Shichao Song, Zifan Zheng, Hanyu Wang, Qingchen Yu, Xunkai Li, Rong-Hua Li, Yi Wang, Zhonghao Wang, Feiyu Xiong, et al. Internal consistency and self-feedback in large language models: A survey. *arXiv preprint arXiv:2407.14507*, 2024.

Renjie Liao, Raquel Urtasun, and Richard Zemel. A pac-bayesian approach to generalization bounds for graph neural networks. *arXiv preprint arXiv:2012.07690*, 2020.

Sanae Lotfi, Marc Finzi, Sanyam Kapoor, Andres Potapczynski, Micah Goldblum, and Andrew G Wilson. Pac-bayes compression bounds so tight that they can explain generalization. *Advances in Neural Information Processing Systems*, 35:31459–31473, 2022.

Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. Self-refine: Iterative refinement with self-feedback. *Advances in Neural Information Processing Systems*, 36:46534–46594, 2023.

David A McAllester. Pac-bayesian stochastic model selection. *Machine Learning*, 51(1):5–21, 2003.

Hossein Mobahi. Training recurrent neural networks by diffusion. *arXiv preprint arXiv:1601.04114*, 2016.

Hyeonseob Nam, HyunJae Lee, Jongchan Park, Wonjun Yoon, and Donggeun Yoo. Reducing domain gap by reducing style bias. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pp. 8690–8699, 2021.

Giambattista Parascandolo, Alexander Neitz, Antonio Orvieto, Luigi Gresele, and Bernhard Schölkopf. Learning explanations that are hard to vary. *arXiv preprint arXiv:2009.00329*, 2020.

Xingchao Peng, Qinxun Bai, Xide Xia, Zijun Huang, Kate Saenko, and Bo Wang. Moment matching for multi-source domain adaptation. In *Proceedings of the IEEE/CVF international conference on computer vision*, pp. 1406–1415, 2019.

Shiori Sagawa, Pang Wei Koh, Tatsunori B Hashimoto, and Percy Liang. Distributionally robust neural networks for group shifts: On the importance of regularization for worst-case generalization. *arXiv preprint arXiv:1911.08731*, 2019.

Yuge Shi, Jeffrey Seely, Philip HS Torr, Narayanaswamy Siddharth, Awni Hannun, Nicolas Usunier, and Gabriel Synnaeve. Gradient matching for domain generalization. *arXiv preprint arXiv:2104.09937*, 2021.

Baochen Sun and Kate Saenko. Deep coral: Correlation alignment for deep domain adaptation. In *European conference on computer vision*, pp. 443–450. Springer, 2016.

Vladimir N Vapnik. The nature of statistical learning. *(No Title)*, 1998.

Hemanth Venkateswara, Jose Eusebio, Shayok Chakraborty, and Sethuraman Panchanathan. Deep hashing network for unsupervised domain adaptation. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 5018–5027, 2017.

Jindong Wang, Cuiling Lan, Chang Liu, Yidong Ouyang, Tao Qin, Wang Lu, Yiqiang Chen, Wenjun Zeng, and Philip S Yu. Generalizing to unseen domains: A survey on domain generalization. *IEEE transactions on knowledge and data engineering*, 35(8):8052–8072, 2022.

Chongzhi Zhang, Mingyuan Zhang, Shanghang Zhang, Daisheng Jin, Qiang Zhou, Zhongang Cai, Haiyu Zhao, Xianglong Liu, and Ziwei Liu. Delving deep into the generalization of vision transformers under distribution shifts. In *Proceedings of the IEEE/CVF conference on Computer Vision and Pattern Recognition*, pp. 7277–7286, 2022.

Hongyi Zhang, Moustapha Cisse, Yann N Dauphin, and David Lopez-Paz. mixup: Beyond empirical risk minimization. *arXiv preprint arXiv:1710.09412*, 2017.

Marvin Zhang, Henrik Marklund, Nikita Dhawan, Abhishek Gupta, Sergey Levine, and Chelsea Finn. Adaptive risk minimization: Learning to adapt to domain shift. *Advances in Neural Information Processing Systems*, 34:23664–23678, 2021.

Xingxuan Zhang, Renzhe Xu, Han Yu, Yancheng Dong, Pengfei Tian, and Peng Cui. Flatness-aware minimization for domain generalization. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 5189–5202, 2023a.

Xingxuan Zhang, Renzhe Xu, Han Yu, Hao Zou, and Peng Cui. Gradient norm aware minimization seeks first-order flatness and improves generalization. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 20247–20257, 2023b.

Kaiyang Zhou, Yongxin Yang, Yu Qiao, and Tao Xiang. Domain generalization with mixstyle. *arXiv preprint arXiv:2104.02008*, 2021.

Juntang Zhuang, Boqing Gong, Liangzhe Yuan, Yin Cui, Hartwig Adam, Nicha Dvornek, Sekhar Tatikonda, James Duncan, and Ting Liu. Surrogate gap minimization improves sharpness-aware training. *arXiv preprint arXiv:2203.08065*, 2022.

A Appendix

A.1 Theoretical Analysis and Intuition (Motivational)

Reader's note. The analysis below was the *motivating intuition* for the HyperDG design: a per-domain Lorentzian formulation that would, by curvature consistency, extend flat-minima robustness to manifold-level invariance. As the controlled multi-seed experiments in Section 4 show, this intuition does *not* translate into a measurable held-out accuracy gain over ERM on standard DG benchmarks in our setting. We retain the theoretical material here because it remains useful as a description of why the conjecture was plausible to begin with and where its assumptions appear to break down empirically. None of the bounds below are claimed to predict the empirical results; the connection between the geometric quantities (curvature spread, sharpness in tangent space) and held-out accuracy was the central hypothesis we tested, and the test came back negative.

In Euclidean learning theory, generalization correlates with the *flatness* of minima in parameter space—models lying in wide basins exhibit low sensitivity to perturbations and unseen data shifts. In the Lorentzian formulation of HyperDG, curvature was intended to play an analogous role but at a higher geometric level: defining the *shape of the feature manifold* rather than the shape of the loss basin. The motivating hypothesis was that ensuring *curvature consistency across domains* would extend flat–minima robustness to manifold–level invariance.

A.2 Riemannian Generalization Bound

Let $\mathcal{M}_i = (\mathbb{L}^{d,1}_{K_i}, g_{K_i})$ denote the Lorentz manifold of domain i with curvature K_i and metric tensor g_{K_i}. For any hypothesis $f : \mathcal{M}_i \to \mathcal{Y}$, the expected risk is

$$\mathcal{R}_{\mathcal{M}_i}(f) = \mathbb{E}_{(x,y)\sim P_i}\big[L(f(x), y)\big], \tag{6}$$

and its empirical counterpart $\hat{\mathcal{R}}_{\mathcal{M}_i}(f)$ is estimated from samples on $\mathcal{M}_i$. Following the Riemannian PAC–Bayes analysis Liao et al. (2020), the generalization gap satisfies

$$\big|\mathcal{R}_{\mathcal{M}_i}(f) - \hat{\mathcal{R}}_{\mathcal{M}_i}(f)\big| \leq \mathcal{O}\left(\sqrt{\frac{\mathrm{KL}(Q\|P) + d^2_{\mathrm{Riem}}(\mathcal{M}_i, \mathcal{M}_s)}{n_i}}\right), \tag{7}$$

where d_{Riem} is the Riemannian distance between the local manifold $\mathcal{M}_i$ and the shared tangent manifold $\mathcal{M}_s$, and $\mathrm{KL}(Q\|P)$ denotes the parameter–space divergence between posterior Q and prior P.

A.3 Curvature Consistency as Distance Minimization

The Lorentzian distance term in equation 7 depends on curvature discrepancy:

$$d^2_{\mathrm{Riem}}(\mathcal{M}_i, \mathcal{M}_s) \propto \|K_i - K_s\|^2 + \|\nabla g_{K_i} - \nabla g_{K_s}\|^2. \tag{8}$$

Minimizing curvature divergence (L_{curv}) directly *shrinks* this inter–manifold distance, tightening the bound equation 7. Hence, curvature alignment provably reduces the variance term controlling generalization under domain shift. Intuitively, when all domains share similar manifold geometry, transporting features between them becomes nearly isometric and preserves semantics.

A.4 Analogy to Flat–Minima PAC–Bayes

In Euclidean PAC Bayes theory McAllester (2003); Lotfi et al. (2022), flat minima correspond to parameter regions where the loss remains stable under perturbations:

$$\mathcal{L}(\Theta + \epsilon) - \mathcal{L}(\Theta) \approx 0, \qquad \forall \|\epsilon\| < \delta. \tag{9}$$

Small Hessian eigenvalues imply broader flatness and tighter generalization bounds. Our Lorentzian extension replaces the Euclidean Hessian curvature with the *sectional curvature* of the feature manifold:

$$\kappa(u, v) = \frac{\langle R(u, v)v, u\rangle_{\mathbb{L}}}{\|u\|_{\mathbb{L}}^2 \|v\|_{\mathbb{L}}^2 - \langle u, v\rangle_{\mathbb{L}}^2}. \tag{10}$$

Maintaining consistent $\kappa(u, v)$ across domains ensures that feature perturbations along different manifold directions yield comparable geodesic distortions, mirroring the flatness–in–loss–space condition. Hence, *flat–minima alignment* $\Leftrightarrow$ *curvature alignment* in the manifold domain.

A.5 Bound for Multi-Domain Generalization

Combining equation 7 and equation 8, the cross–domain generalization gap of the global model f_{Θ_s} satisfies

$$\mathcal{E}_{\text{FDG}} = \mathbb{E}_i\big[|\mathcal{R}_{\mathcal{M}_u}(f_{\Theta_s}) - \mathcal{R}_{\mathcal{M}_i}(f_{\Theta_i})|\big] \leq \mathcal{O}\left(\sqrt{\frac{\overline{\text{KL}} + \text{Var}(K_i)}{N}}\right), \tag{11}$$

where $\text{Var}(K_i)$ measures curvature inconsistency among domains. Minimizing $\text{Var}(K_i)$ through the curvature refinement step tightens the bound equation 11, guaranteeing more stable cross–manifold transfer.

A.6 Geometric Intuition

(a) **Local Flatness → Global Curvature Consistency:** Flat minima ensure robustness to parameter perturbations, whereas curvature consistency ensures robustness to manifold perturbations—i.e., shifts in data geometry.

(b) **Tangent–Space Aggregation:** Aggregating in $T_o\mathbb{L}^{d,1}$ enforces local isometry among domains, reducing distortion in feature transport and stabilizing optimization trajectories.

(c) **Curvature Refinement Loop:** Iteratively updating K_i drives domains toward a shared meta–manifold where gradients, Hessians, and curvatures co–align—the Lorentzian counterpart of refining toward a shared flat basin.

A.7 Theoretical Takeaway

- **Bound Tightening:** Minimizing curvature variance $\text{Var}(K_i)$ contracts the Riemannian generalization bound equation 11, yielding provably lower domain–shift sensitivity.
- **Unified Perspective:** Flat–minima consistency (Li *et al.*, CVPR 2025) operates in Euclidean parameter space; FedGeoRefine extends this to Lorentzian feature space, achieving *geometry–consistent generalization*.
- **Result:** A model trained with curvature refinement not only stabilizes local optimization but also learns a globally invariant geometry that generalizes naturally to unseen domains.

HyperDG formalizes the intuition that curvature alignment is the manifold level analogue of flatness alignment. By ensuring curvature consistent manifolds across domains, it reduces geodesic distortion under domain shift, yielding tighter theoretical and empirical generalization.

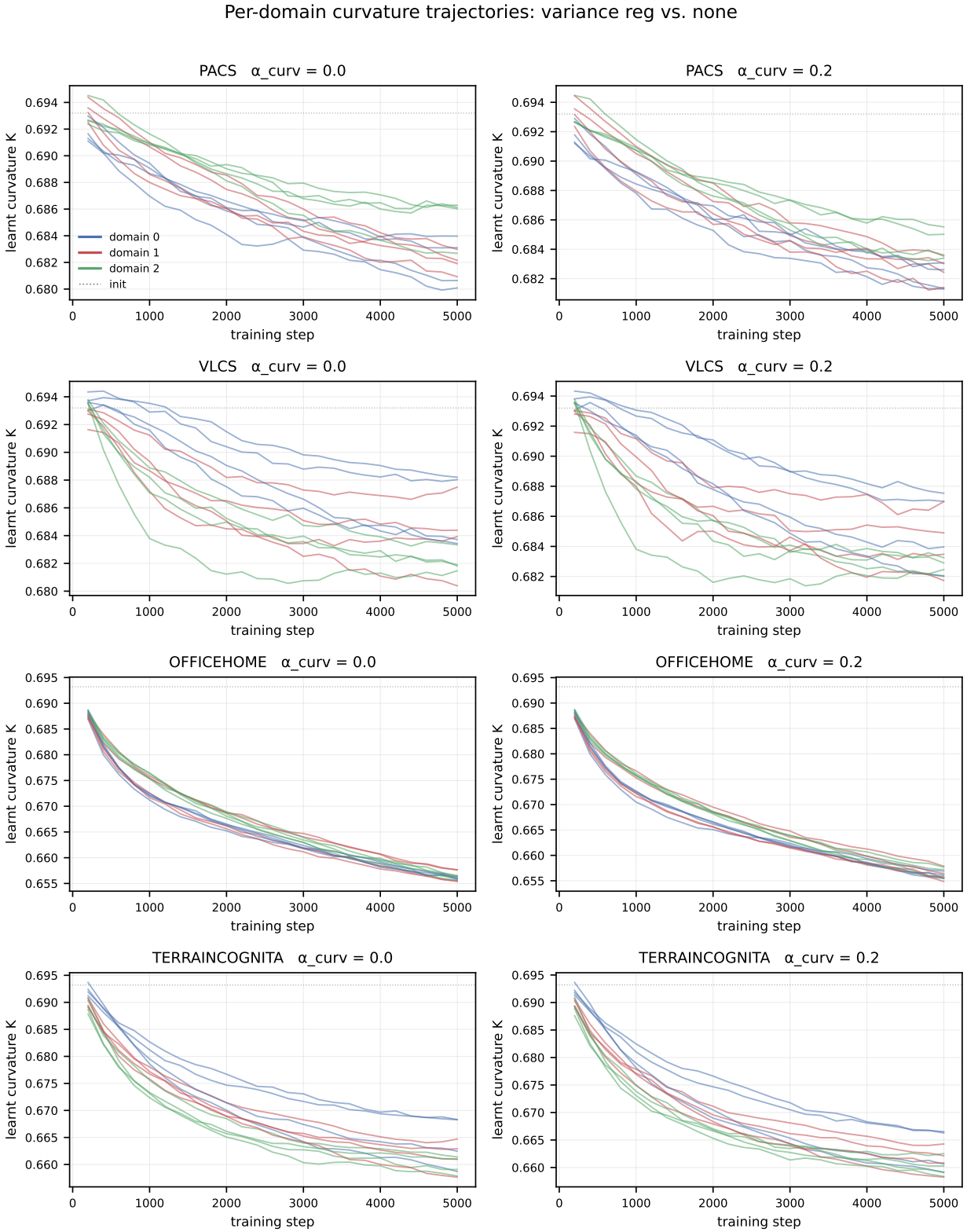


Figure 2: **Per-domain curvature trajectories during HyperDG training, with (right) and without (left) the variance regularizer.** Each panel shows the four leave-one-out splits × three training domains = 12 trajectories. The dashed horizontal line marks the initialization $K_{\text{init}} = \text{softplus}(0) + 10^{-4} \approx 0.6932$. With no penalty, per-domain curvatures still converge to within 0.001–0.011 of one another on every dataset gradient dynamics, not the explicit variance term, drive the alignment. Re-introducing the variance penalty visibly tightens the band only slightly (Table 5). This is the strongest single piece of evidence in the paper that the per-domain curvature parameterization is unnecessary in this setting.

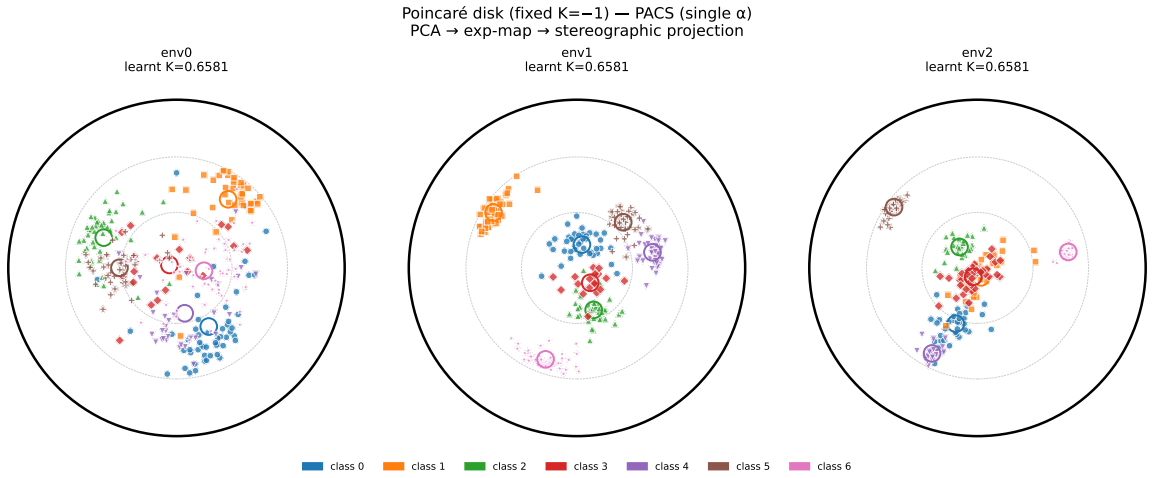


Figure 3: **Per-domain class-prototype Poincaré disks (HyperDG, PACS, split 0).** Each panel is one training domain; each point is one test sample projected via the learned tangent embedding. Color encodes class; rings mark per-class centroids. Clusters are clean and stable across domains the framework *does* learn meaningful structure, but as Figure 4 shows, that structure is angular rather than radial-hierarchical, which is what the per-domain curvature parameterization was supposed to exploit.

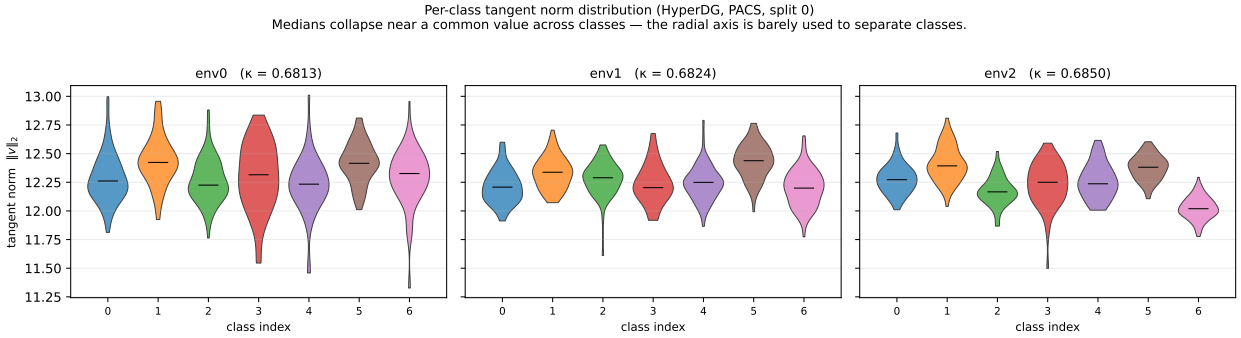


Figure 4: **Per-class tangent-norm distribution (HyperDG, PACS, split 0).** Each violin is the distribution of $\|v\|_2 = \|\text{feats}/\sqrt{K_i}\|_2$ for one class within one training domain (500 samples per class). Medians collapse into a narrow $\sim$0.5-unit band against an absolute scale of $\sim$12, meaning the radial coordinate of the disk is doing essentially no work in separating classes. The classifier discriminates by angular direction the same regime in which a Euclidean classifier on these features would also operate. This is direct visual evidence that the motivating hyperbolic intuition (different classes at different curvature-induced depths) does not materialize in this setting.